

Codici digitali pesati

Unità di Apprendimento 2 · Lezione 1

Tecnologie e Progettazione di Sistemi Informatici e di Telecomunicazioni (TPSI)

Classe 3ª · Istituto Tecnico – Informatica

Prof. Dicara

In questa lezione impareremo

- la codifica dei caratteri **ASCII** e **Unicode** (UTF-8, UTF-16, UTF-32)
- come un calcolatore memorizza più byte: **big-endian** e **little-endian**
- i codici **BCD**

> Filo conduttore: *come si rappresentano numeri e testi con soli 0 e 1, e come si può rendere questa rappresentazione più comoda o più sicura.*

Mappa della lezione

Parte	Argomento
A	Concetti base: codice, alfabeto, lunghezza, ridondanza
B	Codifica dei caratteri: ASCII, ASCII esteso
C	Unicode: UTF-8, UTF-16, UTF-32 e ordine dei byte (endianness)
D	Codici decimali: BCD

Parte A

Codifica dell'informazione: RIPASSO

Perché il calcolatore usa il codice binario? RIPASSO

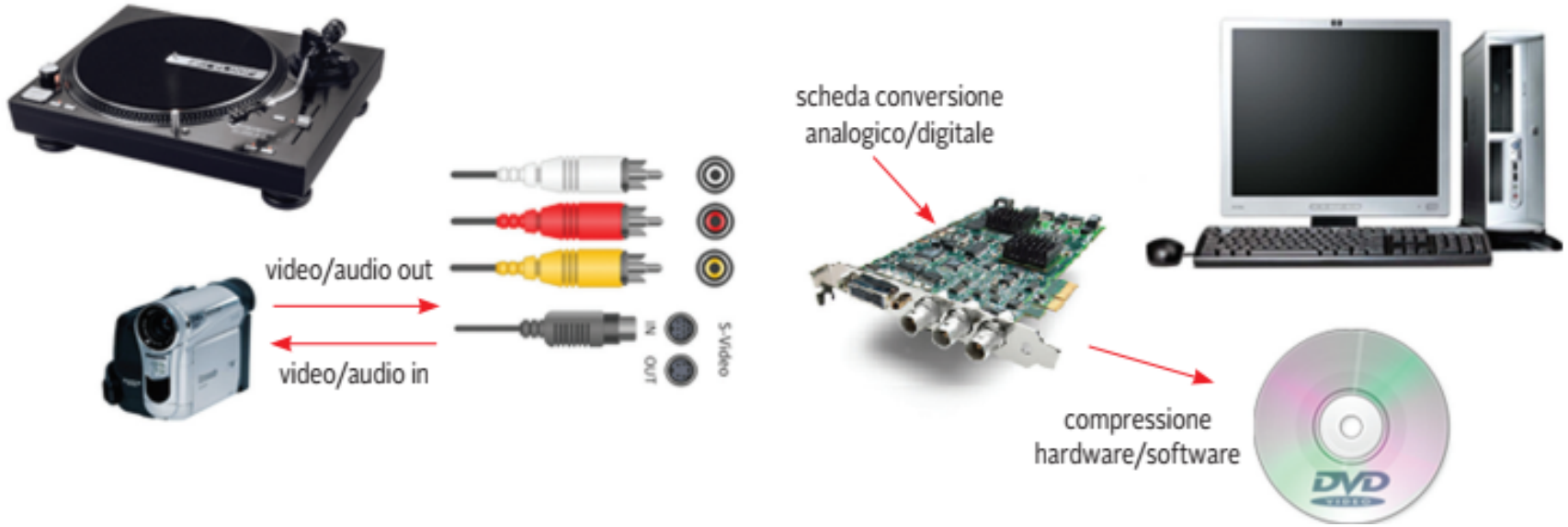
Un calcolatore elettronico elabora **esclusivamente numeri sotto forma di valori digitali**.

Ogni dato (testo, immagine, suono, video) deve essere trasformato in **sequenze di segnali a due valori**, cioè **binari**.

- Un circuito elettronico distingue facilmente **due livelli di tensione**: ad esempio *basso* ≈ 0 V → **0** e *alto* ≈ 5 V (o 3,3 V) → **1**.
- Distinguere due livelli è molto più affidabile che distinguerne dieci: il rumore elettrico non rovina il dato.
- L'unità minima di informazione è il **bit** (*binary digit*); 8 bit formano un **byte**.

> Con **n bit** si ottengono $\mathbf{2^n}$ configurazioni diverse: 8 bit → 256, 16 bit → 65 536, 32 bit → oltre 4 miliardi.

Dal mondo analogico al digitale RIPASSO



Una **scheda di conversione analogico/digitale** acquisisce il segnale di un giradischi o di una videocamera e lo trasforma in dati che il PC può memorizzare (ed eventualmente comprimere, per esempio su DVD).

Le fasi principali sono tre:

1. **Campionamento**: si misura il segnale a intervalli di tempo regolari;

Codice, alfabeto e stringa RIPASSO

- Si dice **codice** una regola che consente di rappresentare un dato insieme di simboli mediante un altro insieme di simboli.
- Si definisce **alfabeto** (indicato con **Σ**) l'insieme *finito* di simboli (caratteri) distinti.
- Si definisce **parola** o **stringa** una sequenza finita di simboli (anche ripetuti) di quell'alfabeto.

> **Esempio.** Σ = {a, b, c}. Le stringhe `abc`, `aab`, `cccc` sono parole di Σ. La stringa `abd` **non** lo è, perché `d` non appartiene all'alfabeto.

Alfabeti: alcuni esempi RIPASSO

Alfabeto	Simbolo	Insieme di simboli
Unario	U	{1}
Binario	B	{0, 1}
Inglese	I	{A, B, C, …, Z}
Alfanumerico	AB	{A, B, …, Z, 0, 1, …, 9}
Caratteri giapponesi, arabi, …	—	{ … }

Una stessa parola può appartenere a più alfabeti:

- `1111001101` è una parola di **B** (e anche di **AB**);
- `AB120C` è una parola di **AB** (non di B: contiene lettere);
- `1111111` è una parola di **B** e di **U**.

Sistema di codifica RIPASSO

Dato un alfabeto **A** composto da **a** simboli, un **sistema di codifica** è un metodo per rappresentare gli *a* simboli di A tramite *b* simboli di un altro alfabeto **B**, stabilendo una **corrispondenza biunivoca** tra gli stessi.

Biunivoca significa che:

- a ogni simbolo di A corrisponde **una sola** parola di B (codifica);
- da ogni parola di B si risale **a un solo** simbolo di A (decodifica, senza ambiguità).

> Se la corrispondenza non fosse biunivoca, non potremmo più recuperare il messaggio originale.

Esempio di codifica: alfabeto a 4 simboli RIPASSO

Sia **A = { R, V, G, B }** (4 caratteri) e **B = { 0, 1 }** (2 caratteri).
Vogliamo codificare in B la stringa di A: **B R R V G**.

Associamo a ogni simbolo la sua **posizione**, scritta in binario su 2 bit:

Simbolo	Posizione	Codice
R	0	**00**
V	1	**01**
G	2	**10**
B	3	**11**

Esempio di codifica: risultato RIPASSO

Stringa da codificare: **B R R V G**

B	R	R	V	G
11	00	00	01	10

$$\mathrm{BRRVG}_{\mathrm{A}} = 1100000110_{\mathrm{B}}.$$

Il destinatario, conoscendo la tabella, **decodifica** leggendo la stringa a **gruppi di 2 bit**:
11·00·00·01·10 → B R R V G.

> Nota: il simbolo "B" dell'alfabeto A (blu) non va confuso con l'alfabeto "B" (binario): per questo si usano i pedici A e B.

Codici nella vita di tutti i giorni

TIPO DI CODICE	MODALITÀ DI COSTRUZIONE
Codice fiscale	Algoritmo sui dati anagrafici
Codice di avviamento postale	In base alla zona geografica
Codice di matricola	Progressivo generale
Numero di targa	Progressivo generale
Partita IVA	In base alla provincia e progressiva
Numero di telefono	In base alla zona geografica e progressiva
Codice tessera sanitaria	In base a un algoritmo

I codici non sono solo informatici: sono **regole di costruzione** di identificatori. Un buon codice è **univoco** e, spesso, contiene informazioni (zona geografica, provincia, controllo di validità).

> Esempio: nel **codice fiscale** l'ultimo carattere è di controllo e viene calcolato con un algoritmo sui primi 15: serve a individuare errori di trascrizione.

Lunghezza del codice RIPASSO

Un **codice binario** è la codifica dei simboli di un alfabeto Σ mediante **stringhe di bit**.

> Un **codice binario** è la codifica dei simboli di un alfabeto **Σ** mediante **stringhe di bit**.

Se **C** è la **cardinalità** di Σ (numero di simboli), il numero **n** di bit da usare per codificare tutti i simboli deve essere tale che:

$$C \leq 2^n$$

cioè, **n** deve essere almeno pari a:

$$n \geq \lceil \log_2 (C) \rceil = m$$

Il simbolo $\lceil \rceil$ indica l'**arrotondamento per eccesso** all'intero superiore. Il valore **m** è il **minimo**

Lunghezza del codice: come si calcola RIPASSO

Procedura rapida: si cerca la più piccola potenza di 2 che sia ≥ C.

C (simboli)	potenze di 2	$m = \lceil \log_2 C \rceil$
2	$2^1 = 2$	1
4	$2^2 = 4$	2
10	$2^3 = 8$ (poco), $2^4 = 16$	4
26	$2^4 = 16$ (poco), $2^5 = 32$	5
128	$2^7 = 128$	7
256	$2^8 = 256$	8

Trucco: per 10 cifre decimali servono 4 bit (non 3: 3 bit danno solo 8 combinazioni).

Codice ridondante e non ridondante RIPASSO

M è il minimo numero di bit necessario per codificare Σ.

- se **n = M** il codice usa il numero minimo di bit;
- se **n > M** il codice è **ridondante** (usa più bit del necessario).

> Un codice si dice **non ridondante** se i simboli che vengono codificati sono in numero uguale alle configurazioni ottenibili con le cifre binarie utilizzate.

Attenzione al linguaggio. Il libro chiama "ridondante" anche un codice con **configurazioni inutilizzate**, cioè con $2^n > C$. Sono due sfumature dello stesso concetto: le combinazioni "in più" **non rappresentano alcun simbolo** e possono essere sfruttate, per esempio, per **rilevare errori**.

Esempio 1 · Alfabeto {A, B, …, Z} RIPASSO

- 26 lettere → **C = 26**
- $n \geq \lceil \log_2 26 \rceil = 5$
- Con 5 bit si hanno $2^5 = 32$ configurazioni
- Ne usiamo 26: **6 configurazioni restano inutilizzate**

Il codice è **ridondante** nel senso di "ha più combinazioni dei simboli".

Esempio pratico: A = 00000, B = 00001, …, Z = 11001. Le combinazioni 11010 … 11111 non corrispondono a nessuna lettera.

ESEMPIO 1

Alfabeto = {A, B, C, …, Z}	
C = 26	cardinalità
$n \geq \lceil \log_2 26 \rceil$	numero di bit
n ≥ 5	
m = 5	minimo numero di bit
$2^5 = 32$	**codice ridondante**

Esempio 2 · Cifre decimali {0, …, 9} RIPASSO

- 10 cifre → **C = 10**
- $n \geq \lceil \log_2 10 \rceil = 4$
- Con 4 bit si hanno $2^4 = 16$ configurazioni (nel riquadro compare per errore 2^6)
- Ne usiamo 10: **6 configurazioni inutilizzate** (1010 … 1111)

Questo è proprio il caso del codice **BCD**, che vedremo nella Parte D.

ESEMPIO 2

Alfabeto = {0, 1, 2, …, 9}	
C = 10	cardinalità
$n \geq \lceil \log_2 10 \rceil$	numero di bit
n ≥ 4	
m = 4	minimo numero di bit
$2^6 = 16$	**codice ridondante**

Esempio 3 · Alfabeto {N, E, S, W} RIPASSO

- 4 punti cardinali → **C = 4**
- $n \geq \lceil \log_2 4 \rceil = 2$
- $2^2 = 4$ configurazioni = 4 simboli

Il codice è **NON ridondante**: ogni configurazione corrisponde a un simbolo.

Simbolo	Codice
N	00
E	01
S	10
W	11

ESEMPIO 3

Alfabeto = {N, E, S, W}

C = 4	cardinalità
$n \geq \lceil \log_2 4 \rceil$	numero di bit
n ≥ 2	
m = 2	minimo numero di bit
$2^2 = 4$	**codice NON ridondante**

Lunghezza fissa o variabile RIPASSO

Possiamo classificare le **codifiche** anche in **base alla lunghezza** delle singole parole.

- **Codifica a lunghezza fissa**: la lunghezza **L** delle parole codice associate ai simboli dell'alfabeto sorgente è costante.
- **Codifica a lunghezza variabile**: la lunghezza **L** delle parole codice associate ai simboli dell'alfabeto sorgente è variabile.

	Lunghezza fissa	Lunghezza variabile
Vantaggi	Decodifica semplice e veloce; accesso diretto al *k*-esimo simbolo	Risparmio di spazio per simboli frequenti
Svantaggi	Possibile spreco di bit	Decodifica più complessa; serve poter capire dove finisce un simbolo
Esempi	ASCII, UTF-32, BCD	UTF-8, UTF-16, codice Morse

Parte B

La codifica dei caratteri: ASCII

Il codice ASCII

ASCII = *American Standard Code for Information Interchange*.
È un codice a **7 bit** → 2^7 = **128** caratteri.

- **0–31 e 127**: caratteri di **controllo** (non stampabili), ad esempio `LF` (a capo), `CR`, `TAB`, `NUL`, `DEL`;
- **32–126**: caratteri **stampabili** (spazio, cifre, lettere maiuscole e minuscole, punteggiatura).

Regole utili da ricordare:

- le cifre `'0'`…`'9'` valgono 48…57 (**0x30…0x39**);
- le maiuscole `'A'`…`'Z'` partono da 65 (**0x41**);
- le minuscole `'a'`…`'z'` partono da 97 (**0x61**): differiscono dalle maiuscole di **32**, cioè dal solo **bit 5**.

La tabella ASCII

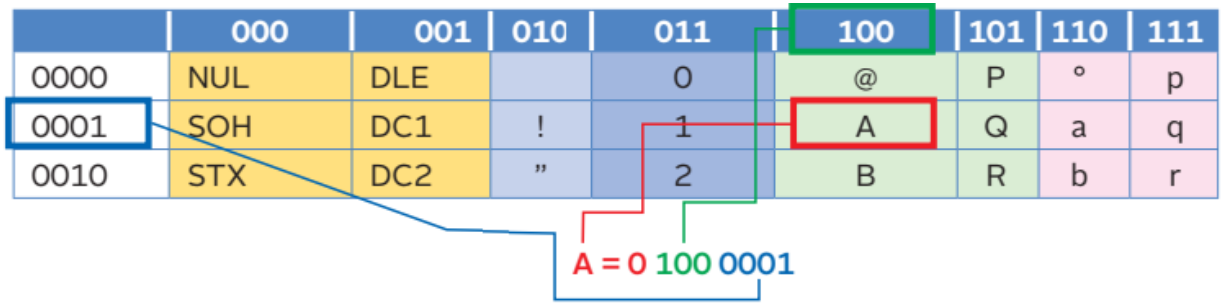
	000	001	010	011	100	101	110	111
0000	NUL	DLE		0	@	P	°	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	w
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	|
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Le **colonne** indicano i 3 bit più significativi, le **righe** i 4 bit meno significativi. *(Nella riga 1001,*

Leggere la tabella ASCII

Codifichiamo il carattere **"A"** in ASCII (su 8 bit):

- il **primo bit** è **0**, perché il codice è a 7 bit;
- i bit 2-4 sono nell'intestazione di **colonna**: **100**;
- i bit 5-8 sono nell'intestazione di **riga**: **0001**.



Otteniamo:

$$\mathbf{A = 0\ 100\ 0001_2 = 41_{16} = 65_{10}}$$

(Il risultato corretto in decimale è 65, non 64: 4·16 + 1 = 65.)

Decodificare una sequenza di bit

ESEMPI

Esempio 1

Decodifichiamo la seguente sequenza di bit:

01001000011001010110110001101100011011110010111 0

Spezziamo la stringa binaria in gruppi da 8 bit; decodifichiamo ogni singolo gruppo assegnando il corrispondente carattere nella tabella ASCII:

01001000 01100101 01101100 01101100 01101111 00101110

01001000	01100101	01101100	01101100	01101111	00101110
H	e	l	l	o	.

Metodo:

1. si suddivide la stringa in **gruppi da 8 bit** (byte);
2. si cerca ogni byte nella tabella ASCII;
3. si scrive il carattere corrispondente.

`01001000 01100101 01101100 01101100 01101111 00101110` → **H e l l o .**

Seconda prova. `01101001 01101100 00000000 01110000 01101111 00101110` → **i, l, NUL, p, o, .** Il byte `00000000` è il carattere di controllo **NUL**: non stampabile.

Prova adesso! · Esercizi ASCII



Prova adesso!

- Codice ASCII esteso



PRENDI CARTA E PENNA

Codifica le seguente scritta in codice ASCII esteso:

<paolo>	<@hoepli.it>

Decodifica le seguenti parole dal codice ASCII:

01100011 01101001 01000000 01101111 01011111 01101101 01000000 01110010 01100101

01101001 01101110 01100110 01101111 01110010 01101101 01100001 01110100 01101001 01100011 01100001

01010111 00100000 01101100 01100001 00100000 01110011 01110001 01110101 01101111 01101100 01100001

Prendi carta e penna: le soluzioni sono alla fine della lezione.

ASCII esteso (Extended ASCII)

Dalla specifica originale a 7 bit si è passati a un'estensione a **8 bit**.

- Scopo: **raddoppiare** i caratteri rappresentabili (da 128 a **256**).
- Il nuovo codice si chiama comunemente **Extended ASCII** o **ASCII estesa**.
- Sono state aggiunte le **vocali accentate**, i **simboli semigrafici** e altri simboli meno comuni.

I primi 128 caratteri **restano identici** all'ASCII originale (compatibilità all'indietro). I caratteri 128–255 cambiano da una variante all'altra.

Le codepage

Le diverse codifiche Extended ASCII sono chiamate **codepage** (pagine di codifica).

- I sistemi operativi devono essere **indipendenti dalla codifica**.
- Per passare da una regione geografica a un'altra basta **sostituire la pagina di codifica**.

Codepage	Uso
ISO-8859-1 (*Latin-1*)	Europa occidentale
ISO-8859-15	come Latin-1, ma con il simbolo dell'euro **€**
Windows-1252	variante Microsoft
ISO-8859-5	alfabeto cirillico

> **Problema classico (*mojibake*):** se un file in UTF-8 viene letto come Latin-1, la lettera "è" (byte `C3 A8`) appare come "Ã¨". Stessi byte, codepage sbagliata!

Parte C

Il codice Unicode

Perché serve Unicode

L'ASCII esteso non basta: 256 caratteri non coprono **cinese, giapponese, arabo, emoji…**, e ogni codepage è incompatibile con le altre.

Unicode nasce per avere **un solo standard** che rappresenti i caratteri di *tutte* le lingue.

- Si basa sulla codifica dell'ASCII esteso (i primi 256 caratteri coincidono con Latin-1);
- invece degli 8 bit dell'ASCII esteso (256 caratteri) usa un codice a **32 bit**.

Un codepoint assume valori da U-0x00000000 a U-0xFFFFFFFF.

Codepoint e UCS

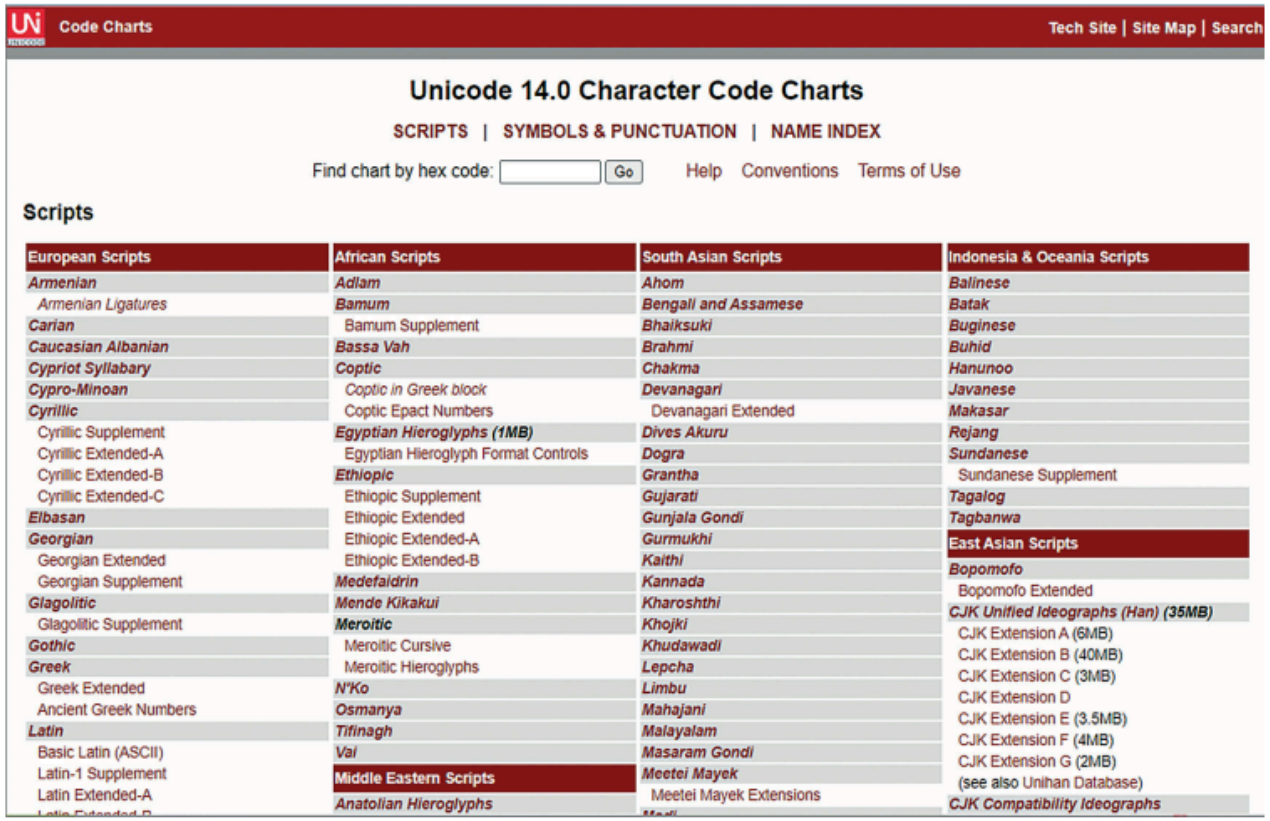
Ogni simbolo è identificato da un numero chiamato **codepoint**, descritto con la notazione **U+*numero esadecimale***.

Carattere	Codepoint	Descrizione
A	U+0041	lettera latina maiuscola A
è	U+00E8	lettera latina e minuscola con accento grave
€	U+20AC	simbolo dell'euro
😀	U+1F600	emoji "faccina sorridente"

L'insieme dei caratteri così ottenuto è l'**UCS** (*Universal Character Set*).

> Nella pratica lo spazio dei codepoint è oggi limitato a **U+0000 … U+10FFFF** (1 114 112 posizioni, 17 piani da 65 536).

Le tabelle ufficiali Unicode



Code Charts — Tech Site | Site Map | Search

Unicode 14.0 Character Code Charts

SCRIPTS | SYMBOLS & PUNCTUATION | NAME INDEX

Find chart by hex code: [ ] Go Help Conventions Terms of Use

Scripts

European Scripts	African Scripts	South Asian Scripts	Indonesia & Oceania Scripts
Armenian	Adlam	Ahom	Balinese
Armenian Ligatures	Bamum	Bengali and Assamese	Batak
Carian	Bamum Supplement	Bhaiksuki	Buginese
Caucasian Albanian	Bassa Vah	Brahmi	Buhid
Cypriot Syllabary	Coptic	Chakma	Hanunoo
Cypro-Minoan	Coptic in Greek block	Devanagari	Javanese
Cyrillic	Coptic Epact Numbers	Devanagari Extended	Makasar
Cyrillic Supplement	Egyptian Hieroglyphs (1MB)	Dives Akuru	Rejang
Cyrillic Extended-A	Egyptian Hieroglyph Format Controls	Dogra	Sundanese
Cyrillic Extended-B	Ethiopic	Grantha	Sundanese Supplement
Cyrillic Extended-C	Ethiopic Supplement	Gujarati	Tagalog
Elbasan	Ethiopic Extended	Gunjala Gondi	Tagbanwa
Georgian	Ethiopic Extended-A	Gurmukhi	**East Asian Scripts**
Georgian Extended	Ethiopic Extended-B	Kaithi	Bopomofo
Georgian Supplement	Medefaidrin	Kannada	Bopomofo Extended
Glagolitic	Mende Kikakui	Kharoshthi	CJK Unified Ideographs (Han) (35MB)
Glagolitic Supplement	Meroitic	Khojki	CJK Extension A (6MB)
Gothic	Meroitic Cursive	Khudawadi	CJK Extension B (40MB)
Greek	Meroitic Hieroglyphs	Lepcha	CJK Extension C (3MB)
Greek Extended	N'Ko	Limbu	CJK Extension D
Ancient Greek Numbers	Osmanya	Mahajani	CJK Extension E (3.5MB)
Latin	Tifinagh	Malayalam	CJK Extension F (4MB)
Basic Latin (ASCII)	Vai	Masaram Gondi	CJK Extension G (2MB)
Latin-1 Supplement	**Middle Eastern Scripts**	Meetei Mayek	(see also Unihan Database)
Latin Extended-A	Anatolian Hieroglyphs	Meetei Mayek Extensions	CJK Compatibility Ideographs

Il sito del **Unicode Consortium** (*Code Charts*) elenca tutti gli *script* (alfabeti) suddivisi per aree geografiche: europei, africani, asiatici…

Dal codepoint ai byte: le codifiche UTF

Il codepoint dice **quale** carattere è; la codifica **UTF** (*UCS Transformation Format*) dice **come scriverlo in byte**.

Sono state definite tre sottocodifiche:

Codifica	Lunghezza	Uso tipico
UTF-8	variabile (1–4 byte)	Web, file di testo, Linux, e-mail
UTF-16	variabile (2 o 4 byte)	elaborazioni applicative: Java, Windows, JavaScript
UTF-32	**fissa**, 4 byte	uso interno quando serve accesso diretto ai caratteri

UTF-8: l'idea

- I primi **128** codepoint (U+0000…U+007F) corrispondono **uno a uno** ai caratteri ASCII → **un solo byte** con il bit più alto a 0.
- I successivi codepoint (fino a 2^{11}) usano **2 byte** e coprono le estensioni ASCII, l'alfabeto greco, cirillico, arabo…
- I codepoint più alti usano **3 o 4 byte**: ideogrammi cinesi, giapponesi, emoji e altre lingue.

Vantaggio: un file di solo testo ASCII è **già** un file UTF-8 valido, quindi la compatibilità con il passato è perfetta.

UTF-8: regola dei byte

I codepoint **maggiori di 127** sono rappresentati da sequenze di byte di lunghezza da 2 a 4 (le specifiche originali arrivavano a 6), composte da un **byte iniziale** e da uno o più **byte di continuazione**.

- Il **byte iniziale** comincia con tanti **1** quanti sono i byte totali, seguiti da uno **0**; poi iniziano i bit del carattere.
- Ogni **byte di continuazione** comincia sempre con **10** e porta 6 bit del carattere.

> Il prefisso `10` non è un bit di parità: serve a **riconoscere subito** un byte di continuazione, quindi a *risincronizzarsi* se si legge a metà di un carattere.

UTF-8: tabella di codifica per range

RANGE CARATTERI UNICODE	CODE POINT (IN BINARIO)	CODIFICA UTF-8 (IN BINARIO)	NOTE
000000–00007F 128 caratteri	0zzzzzzz sette bit z	0zzzzzzz sette bit z	Range equivalente all'ASCII (inizia 0).
000080–0007FF 1920 caratteri	00000yyy yyzzzzzz tre bit y; due bit y, sei bit z	110yyyyy 10zzzzzz cinque bit y, sei bit z	Primo byte inizia 110, secondo 10.
000800–00FFFF 63 488 caratteri	xxxxyyyy yyzzzzzz 4 bit x, 4 bit y; 2 bit y, 6 bit z	1110xxxx 10yyyyyy 10zzzzzz quattro bit x; sei bit y; sei bit z	Primo byte inizia 1110, gli altri 10.
010000–10FFFF 1 048 576 caratteri	000wwwxx xxxxyyyy yyzzzzzz 3 bit w, 2 bit x; 4 bit x, 4 bit y; 2 bit y, 6 bit z	11110www 10xxxxxx 10yyyyyy 10zzzzzz tre bit w; sei bit x; sei bit y; sei bit z	Primo byte inizia 11110, gli altri 10.

Le lettere **x, y, z, w** indicano i bit del codepoint, scritti da sinistra (più significativi) a destra (meno significativi) dentro lo "stampo" del byte.

UTF-8: tabella estesa (fino a 6 byte)

Bit	Last codepoint	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6
7	U+007F	0xxxxxxx					
11	U+07FF	110xxxxx	10xxxxxx				
16	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx			
21	U+1FFFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx		
26	U+3FFFFF	111110xx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	
31	U+7FFFFFFF	1111110x	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx

Lo standard attuale limita UTF-8 a **4 byte** (fino a U+10FFFF); le righe a 5 e 6 byte sono quelle della specifica storica, mantenute per completezza.

UTF-8: come riconoscere la lunghezza

Guardando **solo il primo byte** si capisce quanti byte compongono il carattere.

Primo byte (binario)	Byte totali	Bit utili
`0xxxxxxx`	1	7
`110xxxxx`	2	5 + 6 = **11**
`1110xxxx`	3	4 + 6 + 6 = **16**
`11110xxx`	4	3 + 6 + 6 + 6 = **21**

Nel caso in cui il primo byte inizi per **110** si deduce che:

- il codice è composto da **2 byte**;
- utilizza **5 + 6 = 11 bit** per codificare il carattere.

UTF-8: esempio con "è" (2 byte)

Carattere: è → codepoint **U+00E8** = `1110 1000`

1. Il codepoint sta in 11 bit (range U+0080…U+07FF): `000 1110 1000` → `00011 101000`
2. Si inseriscono i bit nello stampo `110yyyyy 10zzzzzz`:

stampo	`110yyyyy` `10zzzzzz`
bit del carattere	`00011` · `101000`
risultato	`11000011` `10101000` = **C3 A8**

Verifica in Python:

```
>>> "è".encode("utf-8")
b'\xc3\xa8'
```

UTF-8: esempio con "€" (3 byte)

Carattere: € → codepoint **U+20AC** = `0010 0000 1010 1100` (16 bit)

Stampo a 3 byte: `1110xxxx 10yyyyyy 10zzzzzz`

bit divisi 4 + 6 + 6	`0010` · `000010` · `101100`
risultato	`11100010` `10000010` `10101100` = **E2 82 AC**

Altri esempi: **A** → `41` (1 byte) · **è** → `C3 A8` (2 byte) · 😀 U+1F600 → `F0 9F 98 80` (4 byte).

> **Per esercizio:** codifica "ñ" (U+00F1) in UTF-8. *(Soluzione a fine lezione.)*

Confronto: lo stesso testo in più codifiche

Carattere	Codepoint	UTF-8	UTF-16 BE	UTF-32 BE
A	U+0041	`41`	`00 41`	`00 00 00 41`
è	U+00E8	`C3 A8`	`00 E8`	`00 00 00 E8`
€	U+20AC	`E2 82 AC`	`20 AC`	`00 00 20 AC`
😀	U+1F600	`F0 9F 98 80`	`D8 3D DE 00`	`00 01 F6 00`
byte occupati		1 · 2 · 3 · 4	2 · 2 · 2 · 4	4 · 4 · 4 · 4

UTF-8 è il più compatto per i testi occidentali; UTF-32 è il più semplice ma il più "costoso".

Parte C-bis

Come la RAM memorizza i dati: endianness

La memorizzazione dei dati nella RAM

La memoria RAM è divisa in **unità (locazioni)**; ogni **indirizzo** individua una locazione.

- la locazione ha dimensione standard di **8 bit = 1 byte**;
- l'indirizzo ha come dimensione usuale **32 bit = 4 byte** (nei sistemi a 64 bit: 64 bit).

Indirizzo	0	1	2	3	4	5	…
Contenuto	1 byte	1 byte	1 byte	1 byte	1 byte	1 byte	…

Big-endian e little-endian

I calcolatori operano su **parole di 32–64 bit** (4–8 byte). Per memorizzare una parola su **più locazioni da 1 byte contigue** esistono due possibilità:

- **big-endian (BE)**: il byte **più significativo** (la "testa" del numero) va nell'indirizzo **più basso**;
- **little-endian (LE)**: il byte **meno significativo** va nell'indirizzo **più basso**; il byte più significativo finisce all'indirizzo più alto.

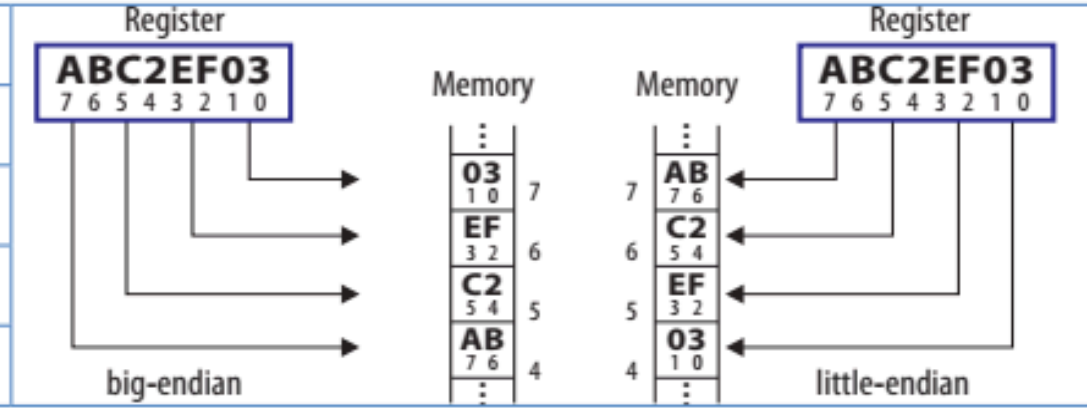
> **Per ricordare:** *big-endian* = "il grande (MSB) viene per primo", si legge come scriviamo i numeri. Il **network byte order** (i protocolli di rete) è big-endian; le CPU x86 e la maggior parte delle ARM usano little-endian.

Esempio: la parola AB C2 EF 03

ESEMPIO

Inseriamo la parola "AB C2 EF 03" nei byte con indirizzi da 4 a 7:

BIG-ENDIAN	LITTLE-ENDIAN
$4 \rightarrow AB\ (10101011)_2$	$4 \rightarrow 03\ (00000011)_2$
$5 \rightarrow C2\ (11000010)_2$	$5 \rightarrow EF\ (11101111)_2$
$6 \rightarrow EF\ (11101111)_2$	$6 \rightarrow C2\ (11000010)_2$
$7 \rightarrow 03\ (00000011)_2$	$7 \rightarrow AB\ (10101011)_2$



La parola **0xABC2EF03** viene inserita nei byte con indirizzi da 4 a 7:

Indirizzo	Big-endian	Little-endian
4	AB	03
5	C2	EF
6	EF	C2
7	03	AB

Endianness in pratica (Python)

```python
import sys
print(sys.byteorder)            # 'little' sui PC x86/x64

n = 0xABC2EF03
print(n.to_bytes(4, "big").hex(" "))      # ab c2 ef 03
print(n.to_bytes(4, "little").hex(" "))   # 03 ef c2 ab
```

Per questo i file di testo UTF-16 e UTF-32 iniziano spesso con un **BOM** (*Byte Order Mark*, U+FEFF):

- `FE FF` → big-endian
- `FF FE` → little-endian

UTF-16: i piani Unicode

Lo spazio di codice Unicode è diviso in **17 "piani"** di 2^{16} = **65 536** codepoint ciascuno.

- I codepoint di ogni piano hanno valori esadecimali da **xx0000** a **xxFFFF**.
- **xx** è un valore esadecimale **00–10** che indica il **piano** di appartenenza; la parte restante si ripete nei vari piani.
- Il **piano 0**, quello dei codepoint U+0000…U+FFFF, si chiama **BMP** (*Basic Multilingual Plane*): contiene quasi tutte le lingue di uso comune.

Regola UTF-16:

- codepoint nel **BMP** → **1 unità da 16 bit** (2 byte);
- codepoint **oltre** il BMP → **2 unità da 16 bit** (*surrogate pair*, 4 byte).

UTF-16 · Esempio 1: la lettera "A"

Codepoint (UCS a 32 bit): **U+0x00000041**

`0000-0000-0000-0000-0000-0000-0100-0001`

I 16 bit più significativi sono tutti a 0 → ricade nel **BMP** → UTF-16: **0x0041** (`0000-0000-0100-0001`).

Si ottengono due rappresentazioni, a seconda dell'ordine dei byte:

Codifica	Byte in memoria
UTF-16BE	`00 41`
UTF-16LE	`41 00`

UTF-16 · Esempio 2: la lettera "è"

Codepoint (UCS a 32 bit): **U+0x000000E8**

`0000-0000-0000-0000-0000-0000-1110-1000`

16 bit alti a 0 → **BMP** → UTF-16: **0x00E8** (`0000-0000-1110-1000`).

Codifica	Byte in memoria
UTF-16BE	`00 E8`
UTF-16LE	`E8 00`

> **Attenzione:** "è" (accento grave) è U+00E8; "é" (accento acuto) è U+00E9. Sono due caratteri diversi!

UTF-16 · Esempio 3: il simbolo "€"

Codepoint (UCS a 32 bit): **U+0x000020AC**

`0000-0000-0000-0000-0010-0000-1010-1100`

16 bit alti a 0 → **BMP** → UTF-16: **0x20AC** (`0010-0000-1010-1100`).

Codifica	Byte in memoria
UTF-16BE	`20 AC`
UTF-16LE	`AC 20`

UTF-16 · Oltre il BMP: surrogate pair

Per un codepoint **> U+FFFF** (per esempio l'emoji 😀, U+1F600):

1. si sottrae 0x10000: `0x1F600 – 0x10000 = 0xF600`
2. si prendono i **10 bit alti** (0x3D) e i **10 bit bassi** (0x200);
3. **high surrogate** = 0xD800 + 0x3D = **0xD83D**;
   low surrogate = 0xDC00 + 0x200 = **0xDE00**.

Risultato: 😀 = **D83D DE00** (UTF-16BE: `D8 3D DE 00`).

> Per questo i valori da D800 a DFFF **non** sono assegnati a caratteri: sono riservati ai surrogati.

UTF-32

UTF-32 è una codifica a **lunghezza fissa di 4 byte**.

- I codici dei caratteri **coincidono con i codici UCS**.
- Consente la **rappresentazione contemporanea** dei codici di tutte le lingue.
- Anche in UTF-32 sono possibili le due varianti **little-endian** e **big-endian**.

Carattere	UTF-32BE	UTF-32LE
A	`00 00 00 41`	`41 00 00 00`
€	`00 00 20 AC`	`AC 20 00 00`

Pro: il k-esimo carattere sta sempre all'offset $4 \cdot k$. **Contro:** uno spreco di memoria enorme per testi latini (3 byte su 4 sono zeri).

Parte D

Codici decimali: BCD

Il codice BCD (Binary Coded Decimal)

Un sistema che permette di rappresentare **esclusivamente le dieci cifre decimali** è la codifica **BCD** (*Binary Coded Decimal*, "decimale codificato in binario").

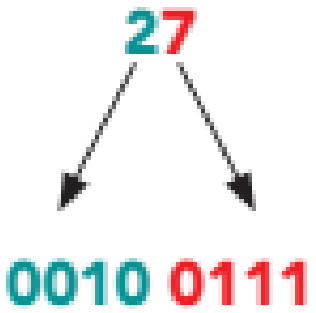
NUMERO DECIMALE	NUMERO BCD			
peso →	8	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Ogni cifra decimale 0–9 è scritta su **4 bit** con i **pesi 8-4-2-1** (come il binario naturale). Si ottengono 10 combinazioni valide (0000–1001); le altre 6 (**1010–1111**) **non sono ammesse**.

BCD: codificare un numero

Si **codifica ogni cifra separatamente**, su 4 bit.

- Codifica in codice BCD il numero $(127)_{10} = (0001\ 0010\ 0111)_{BCD}$
- Codifica in codice BCD il numero $(379)_{10} = (0011\ 0111\ 1001)_{BCD}$



> Esempio passo passo per $(305)_{10}$: 3 → `0011`, 0 → `0000`, 5 → `0101` → $(0011\ 0000\ 0101)_{BCD}$

BCD non è binario!

Non bisogna confondere il codice BCD con la codifica dei numeri nel sistema binario: i tre esempi riportati codificati in binario sono:

$27_{10} = 11011_2 = 0010\ 0111_{BCD}$

$127_{10} = 1111111_2 = 0001\ 0010\ 0111_{BCD}$

$379_{10} = 101111011_2 = 0011\ 0111\ 1001_{BCD}$

Il codice BCD codifica singolarmente ogni cifra, non l'insieme delle cifre: quindi solo le prime dieci configurazioni della codifica BCD coincidono con la codifica binaria.

Il codice BCD codifica **singolarmente ogni cifra**, non l'insieme delle cifre: solo le prime dieci configurazioni coincidono con la codifica binaria.

Decimale	Binario puro	BCD
9	1001	1001
10	1010	0001 0000
27	11011	0010 0111
127	1111111	0001 0010 0111

Dove si usa? Display a 7 segmenti, orologi digitali, calcolatrici, sistemi finanziari: dove si deve